

DAY-7 — Bash Scripting Basics

CyberNotes • Detailed Beginner Reference (Improved Edition)

Learning goal: Build a practical, working understanding of **bash scripting basics**, connect the concept to real cybersecurity work, and practice it safely in a local lab.

Study rule: Understand a command or concept before copying it. Security tools are powerful because they interact with real systems — your practice environment should always be intentionally controlled and authorized.

1. Core Concepts

Core idea: Bash Scripting Basics is a building block for practical cybersecurity. This section explains what each term actually means, where you'll encounter it, and how it connects to the bigger picture.

Why it matters: Security work depends on understanding normal system and network behavior before you can recognize abnormal behavior. This topic connects directly to Linux, networking, web security, monitoring, and troubleshooting — the threads running through this whole 20-day series.

Vocabulary covered: variables, quoting, arguments, conditions, loops, functions, exit codes, safe scripts.

Reference	What to remember
<code>bash script.sh</code>	run script
<code>chmod +x script.sh</code>	make executable
<code>\$1</code>	first argument
<code>\$?</code>	last exit code
<code>if/elif/else</code>	conditions
<code>for/while</code>	loops
<code>function name(){ }</code>	function

2. Concept Explanations

Each term below is explained in plain language, with a note on why it matters for security work. Read these before running any commands.

1. variables

A variable stores a value for reuse, assigned with ``name=value`` (no spaces around ``=``) and read with ``$name``. Scripts commonly use variables to hold things like a target host or a file path so the same logic can run against different inputs without editing the script.

2. quoting

Quoting controls how bash treats spaces and special characters. Double quotes (`"$var"`) let variables expand but protect spacing; single quotes (`'$var'`>) prevent expansion entirely. Forgetting to quote a variable`

that might contain a space or a wildcard is one of the most common — and sometimes dangerous — bash mistakes, especially in scripts that build file paths or commands from user input.

3. arguments

Arguments are values passed to a script when it's run, accessible as ``$1``, ``$2``, etc., with ``$0`` being the script's own name and ``$@`` all arguments together. This is what makes a script reusable rather than hard-coded to one specific target or file.

4. conditions

``if`/`elif`/`else`` let a script branch based on a test, such as checking whether a file exists (``[ -f file ]``) or whether a command succeeded. Conditions are what turn a script from a fixed sequence of commands into logic that adapts to its environment.

5. loops

``for`` loops iterate over a known list (like a set of hostnames), while ``while`` loops repeat as long as a condition holds true (like reading a file line by line). Loops are essential for automating repetitive tasks, such as running the same check across many hosts.

6. functions

A function groups a block of commands under a name so it can be called multiple times without repeating code, defined as ``function name(){ ...; }`` or simply ``name(){ ...; }``. Functions make scripts easier to read, test, and reuse.

7. exit codes

Every command returns a numeric exit code when it finishes: ``0`` means success, any non-zero value means some kind of failure, retrievable via ``$?``. Checking exit codes lets a script react to failure instead of blindly continuing — critical when a script is doing anything security-sensitive.

8. safe scripts

A safe script validates its inputs, quotes its variables, checks exit codes before proceeding, and avoids destructive commands (like ``rm -rf``) without explicit confirmation. Options like ``set -euo pipefail`` at the top of a script make it fail fast and loud instead of silently continuing after an error — a habit worth building early.

3. Safe Practical Lab

Step 1. Define the lab target and the exact question you are trying to answer.

Step 2. Run the smallest safe observation or command that can answer that question.

Step 3. Read the output carefully; record important fields, errors, timestamps, and assumptions.

Step 4. Change one variable at a time and compare the result.

Step 5. Write a short conclusion and clean up temporary files or services.

Example lab sequence:

```
printf '#!/bin/bash\nnecho "Hello $USER"\n' > hello.sh
chmod +x hello.sh
bash hello.sh
echo $?
```

4. Common Mistakes & Troubleshooting

- Running a command without checking its target, current directory, or privileges.
- Confusing a tool's output with proof of a vulnerability or compromise — output is evidence, not a conclusion.
- Ignoring error messages instead of reading the exact reason for failure.
- Changing several settings at once, making it hard to tell what actually fixed or broke the lab.
- Saving real credentials, tokens, or personal data in practice notes or public repositories.
- Testing a public target just because it's reachable — reachability is never the same as authorization.

Troubleshooting checklist:

- What system am I on?
- What exact command/request did I run?
- What did I expect?
- What happened instead?
- What does the error message say?
- Can I reproduce it in a clean local lab?
- What changed since the last successful attempt?

5. Deep-Dive Questions

Answer these in your own words after finishing the lab — they're designed to check real understanding, not memorization.

Q1. What problem does variables solve, in your own words?

Notes: _____

Q2. What is the difference between normal behavior and a suspicious observation in this topic?

Notes: _____

Q3. Which command, log, packet, file, or response would you inspect first, and why?

Notes: _____

Q4. What could cause a misleading result or false positive here?

Notes: _____

Q5. How would you document your observation so another learner could reproduce it safely?

Notes: _____

Q6. Which security control reduces the risk associated with this topic?

Notes: _____

6. Revision Checklist

- I can explain bash scripting basics in my own words.
- I can identify the main terms without looking at the notes.
- I can perform the example in a controlled lab.
- I can interpret the output instead of only copying it.
- I know at least one defensive or troubleshooting use for the concept.
- I can document the result clearly and safely.

7. Safety & Scope

Use these commands and tools only on systems you own, a local virtual machine, a CTF, an intentionally vulnerable application, or a target for which you have explicit written permission.

Keep a written scope: target, allowed actions, time window, and stop conditions. Never scan or test random public systems.

When practicing, save your notes and outputs so you can explain what you observed rather than blindly running commands.

Next step: After Day-7, review your notes and connect this topic to the next subject in the series. The goal is not to memorize commands — it's to understand systems well enough to reason about security.

8. Quick Reference & Personal Notes

Topic: Day-7 — Bash Scripting Basics

Three things I learned:

1. _____
1. _____
1. _____

Commands / concepts to memorize:

- `bash script.sh` — run script
- `chmod +x script.sh` — make executable
- `$1` — first argument
- `$?` — last exit code
- `if/elif/else` — conditions
- `for/while` — loops
- `function name(){ }` — function

My lab result

What I tested: _____

What I observed: _____

What I would investigate next: _____

Final reminder: A strong cybersecurity learner can explain both the offensive concept and the defensive reason for learning it. Keep practice authorized, reproducible, and documented.